

Line-by-Line Mathematical Guide for HAETAE_ROM.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Typed random-oracle query and output model.

How To Read This Script

Read this file as the hash-domain interface. It separates the query forms so later freshness and programming proofs can refer to precise events.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore Distr List PROM.</code>	Imports AllCore Distr List PROM. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Params HAETAE_Algebra HAETAE_Distributions.</code>	Imports HAETAE_Params HAETAE_Algebra HAETAE_Distributions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4	<code>theory HAETAE_ROM.</code>	Begins the namespace HAETAE_ROM, grouping the declarations as one checked theory.
5	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6	<code>import HAETAE_Algebra.</code>	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
7	<code>import HAETAE_Distributions.</code>	Opens HAETAE_Distributions so its names can be used without qualification in the following proof.
8	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.
9	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
10	<code>type ro_query =</code>	Declares carrier set ro_query. EasyCrypt will type-check all later functions and games over this mathematical domain.
11	<code>[MessageHashQuery of (pkey * context * message)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
12	<code> ChallengeHashQuery of (mode * polyveck * poly * crh)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
13	<code> MatrixExpandQuery of (mode * seed)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
14	<code> SamplerExpandQuery of random_coins].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
15	<code>type ro_output = crh * challenge * matrix * random_coins.</code>	Declares carrier set <code>ro_output</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.
16	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
17	<code>op message_hash_query (pk : pkey) (ctx : context) (m : message) : ro_query =</code>	Defines operation <code>message_hash_query</code> , a total mathematical function or abbreviation used by later declarations.
18	<code>MessageHashQuery (pk, ctx, m).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
19	<code>op challenge_hash_query</code>	Defines operation <code>challenge_hash_query</code> , a total mathematical function or abbreviation used by later declarations.
20	<code>(md : mode) (w1 : polyveck) (w0 : poly) (mu : crh) : ro_query =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
21	<code>ChallengeHashQuery (md, w1, w0, mu).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
22	<code>op matrix_expand_query (md : mode) (sd : seed) : ro_query =</code>	Defines operation <code>matrix_expand_query</code> , a total mathematical function or abbreviation used by later declarations.
23	<code>MatrixExpandQuery (md, sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
24	<code>op sampler_expand_query (coins : random_coins) : ro_query =</code>	Defines operation <code>sampler_expand_query</code> , a total mathematical function or abbreviation used by later declarations.
25	<code>SamplerExpandQuery coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
26	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
27	<code>op ro_crh_zero : crh = nseq crhbytes 0.</code>	Defines operation <code>ro_crh_zero</code> , a total mathematical function or abbreviation used by later declarations.
28	<code>op ro_challenge_zero : challenge = poly_zero.</code>	Defines operation <code>ro_challenge_zero</code> , a total mathematical function or abbreviation used by later declarations.
29	<code>op ro_matrix_zero : matrix = [].</code>	Defines operation <code>ro_matrix_zero</code> , a total mathematical function or abbreviation used by later declarations.
30	<code>op ro_random_coins_zero : random_coins = nseq seedbytes 0.</code>	Defines operation <code>ro_random_coins_zero</code> , a total mathematical function or abbreviation used by later declarations.
31	<code>op ro_output_zero : ro_output =</code>	Defines operation <code>ro_output_zero</code> , a total mathematical function or abbreviation used by later declarations.
32	<code>(ro_crh_zero, ro_challenge_zero, ro_matrix_zero, ro_random_coins_zero).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
33	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
34	<code>op ro_output_to_crh (y : ro_output) : crh = y.'1.</code>	Defines operation <code>ro_output_to_crh</code> , a total mathematical function or abbreviation used by later declarations.
35	<code>op ro_output_to_challenge (y : ro_output) : challenge = y.'2.</code>	Defines operation <code>ro_output_to_challenge</code> , a total mathematical function or abbreviation used by later declarations.
36	<code>op ro_output_to_matrix (y : ro_output) : matrix = y.'3.</code>	Defines operation <code>ro_output_to_matrix</code> , a total mathematical function or abbreviation used by later declarations.
37	<code>op ro_output_to_random_coins (y : ro_output) : random_coins = y.'4.</code>	Defines operation <code>ro_output_to_random_coins</code> , a total mathematical function or abbreviation used by later declarations.
38	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
39	<code>op ro_output_of_crh (mu : crh) : ro_output =</code>	Defines operation <code>ro_output_of_crh</code> , a total mathematical function or abbreviation used by later declarations.
40	<code>(mu, ro_challenge_zero, ro_matrix_zero, ro_random_coins_zero).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
41	<code>op ro_output_of_challenge (ch : challenge) : ro_output =</code>	Defines operation <code>ro_output_of_challenge</code> , a total mathematical function or abbreviation used by later declarations.
42	<code>(ro_crh_zero, ch, ro_matrix_zero, ro_random_coins_zero).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
43	<code>op ro_output_of_matrix (a : matrix) : ro_output =</code>	Defines operation <code>ro_output_of_matrix</code> , a total mathematical function or abbreviation used by later declarations.
44	<code>(ro_crh_zero, ro_challenge_zero, a, ro_random_coins_zero).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
45	<code>op ro_output_of_random_coins (coins : random_coins) : ro_output =</code>	Defines operation <code>ro_output_of_random_coins</code> , a total mathematical function or abbreviation used by later declarations.
46	<code>(ro_crh_zero, ro_challenge_zero, ro_matrix_zero, coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
47	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
48	op dmatrix_for_query (x : mode * seed) : matrix distr = dmatrix x.'1.	Defines dmatrix_for_query as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
49	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
50	op dro_output (q : ro_query) : ro_output distr =	Defines dro_output as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
51	with q = MessageHashQuery _ => dmap dcrh ro_output_of_crh	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
52	with q = ChallengeHashQuery x =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
53	dmap (dchallenge x.'1) ro_output_of_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	with q = MatrixExpandQuery x =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
55	dmap (dmatrix_for_query x) ro_output_of_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
56	with q = SamplerExpandQuery _ =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
57	dmap drandom_coins ro_output_of_random_coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
58	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
59	lemma matrix_query_distribution_lossless x :	States checked lemma matrix_query_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
60	is_lossless (dmatrix_for_query x).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
61	proof.	Begins the proof script for the preceding proposition.
62	rewrite /dmatrix_for_query.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
63	case x => md sd /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
64	by apply matrix_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
65	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
66	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
67	lemma ro_output_distribution_lossless q :	States checked lemma ro_output_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
68	is_lossless (dro_output q).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	proof.	Begins the proof script for the preceding proposition.
70	rewrite /dro_output.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
71	case q => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
72	+ by apply dmap_ll; apply crh_distribution_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
73	+ move=> x.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
74	by apply dmap_ll; apply challenge_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
75	+ move=> x.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
76	by apply dmap_ll; apply matrix_query_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
77	+ by apply dmap_ll; apply signing_coin_distribution_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
78	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
79	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
80	clone import FullRO as HAETAE_RO with	Clones and opens a parameterized EasyCrypt theory, producing a specialized instance used below.
81	type in_t <- ro_query,	Declares carrier set in_t. EasyCrypt will type-check all later functions and games over this mathematical domain.
82	type out_t <- ro_output,	Declares carrier set out_t. EasyCrypt will type-check all later functions and games over this mathematical domain.

Line	EasyCrypt source	Mathematical reading
83	<code>op dout <- dro_output.</code>	Defines operation <code>dout</code> , a total mathematical function or abbreviation used by later declarations.
84	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
85	<code>module type Oracle = {</code>	Defines module interface <code>Oracle</code> . It specifies the procedures that later games or adversaries may call.
86	<code>include FRO [init, get, set, rem, sample, queried, allKnown]</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
87	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
88	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
89	<code>module type POracle = {</code>	Defines module interface <code>POracle</code> . It specifies the procedures that later games or adversaries may call.
90	<code>include FRO [get, set, rem, sample, queried, allKnown]</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
91	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
92	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
93	<code>op ro_message_hash (y : ro_output) : crh = ro_output_to_crh y.</code>	Defines operation <code>ro_message_hash</code> , a total mathematical function or abbreviation used by later declarations.
94	<code>op ro_challenge_hash (y : ro_output) : challenge = ro_output_to_challenge y.</code>	Defines operation <code>ro_challenge_hash</code> , a total mathematical function or abbreviation used by later declarations.
95	<code>op ro_matrix (y : ro_output) : matrix = ro_output_to_matrix y.</code>	Defines operation <code>ro_matrix</code> , a total mathematical function or abbreviation used by later declarations.
96	<code>op ro_signing_coins (y : ro_output) : random_coins =</code>	Defines operation <code>ro_signing_coins</code> , a total mathematical function or abbreviation used by later declarations.
97	<code>ro_output_to_random_coins y.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
98	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
99	<code>lemma sampler_expand_query_dro_output_ro_signing_coins</code>	States checked lemma <code>sampler_expand_query_dro_output_ro_signing_coins</code> . The following proof script must derive this proposition from prior definitions and lemmas.
100	<code>seed_coins (p : random_coins -> bool) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
101	<code>mu (dro_output (sampler_expand_query seed_coins))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
102	<code>(fun y => p (ro_signing_coins y)) =</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
103	<code>mu drandom_coins p.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
104	<code>proof.</code>	Begins the proof script for the preceding proposition.
105	<code>rewrite /dro_output /sampler_expand_query /ro_signing_coins</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
106	<code>/ro_output_to_random_coins /ro_output_of_random_coins /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
107	<code>rewrite dmapE.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
108	<code>by apply mu_eq => coins; rewrite /(\o) /=.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
109	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
110	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
111	<code>end HAETAE_ROM.</code>	Closes the current theory or module block.